

Informe de Laboratorio

Reglas de Negocio con Spring Boot y Drools

1. Introducción

En el desarrollo de software empresarial es común encontrar procesos que requieren la aplicación de reglas de negocio complejas y cambiantes. Para resolver este problema existen herramientas conocidas como motores de reglas, las cuales permiten separar la lógica del negocio del código principal de la aplicación.

En este laboratorio se trabajó con Drools, un BRMS (Business Rules Management System) basado en Java que permite definir reglas de negocio de manera declarativa mediante archivos DRL. Además, se integró con Spring Boot para construir una aplicación capaz de evaluar solicitudes de crédito bancario según diferentes condiciones como ingresos, edad, historial crediticio y tipo de crédito solicitado.

El uso de Drools facilita la administración de reglas empresariales, permitiendo modificar la lógica del negocio sin alterar directamente el código fuente principal de la aplicación. Asimismo, se implementaron validaciones con Spring Validation, una API REST para recibir solicitudes y una integración con inteligencia artificial para generar explicaciones automáticas de las decisiones tomadas por el sistema.

2. Objetivos

Objetivo General:

Desarrollar una aplicación utilizando Spring Boot y Drools para implementar un sistema de evaluación de solicitudes de crédito basado en reglas de negocio.

Objetivos Específicos:

- Comprender el funcionamiento del motor de reglas Drools.
- Configurar un proyecto Spring Boot con integración de Drools.
- Implementar reglas de negocio utilizando archivos DRL.
- Aplicar validaciones de datos mediante Spring Validation.
- Construir una API REST para evaluar solicitudes de crédito.
- Integrar inteligencia artificial para explicar los resultados generados por las reglas.
- Realizar pruebas funcionales utilizando Postman.

3. Breve Marco Teórico

Drools es un motor de reglas de negocio desarrollado en Java que permite implementar lógica empresarial mediante reglas declarativas. Hace parte del

ecosistema KIE (Knowledge Is Everything) y utiliza una adaptación orientada a objetos del algoritmo Rete para procesar reglas eficientemente.

Spring Boot es un framework basado en Spring que facilita el desarrollo de aplicaciones Java empresariales. Permite configurar aplicaciones rápidamente mediante dependencias y configuraciones automáticas.

Las reglas en Drools se escriben utilizando el lenguaje DRL (Drools Rule Language), compuesto principalmente por una sección when y una sección then.

KIE es la plataforma que agrupa las tecnologías relacionadas con Drools y proporciona herramientas para cargar, construir y ejecutar reglas de negocio.

Spring Validation permite validar automáticamente los datos enviados por el usuario utilizando anotaciones como @NotBlank, @NotNull, @Min y @Size.

4. Procedimiento con los respectivos pantallazos.

- Estructura del Proyecto.

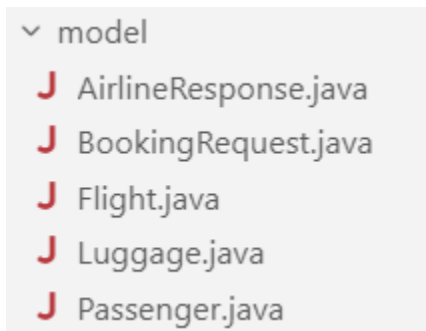


- Configuración de dependencias en Maven.

El BOM de Spring AI es un archivo especial de Maven llamado Bill Of Materials que sirve para administrar automáticamente las versiones de las dependencias de Spring AI. Básicamente, evita que tengas que escribir la versión en cada dependencia individual.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>${spring-ai.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

A su vez también añadimos las dependencias correspondientes a spring boot dev tool, AI, starter web, starter validation, Lombok, drool y kie



1. Passenger — Es el modelo que representa al pasajero y contiene los datos que varias reglas evalúan: name, age, membershipStatus (Basic, Gold, Platinum), travelingWithChildren y seatPreference. Usa anotaciones de Jakarta Validation como @NotBlank, @NotNull, @Size y @Min para validar los datos antes de procesarlos. Tiene constructor vacío y getters/setters, que Spring y Drools necesitan para mapear el formulario y leer propiedades como passenger.age en las reglas.
2. Flight — Modela la información del vuelo con delayMinutes (retraso en minutos) y durationHours (duración en horas). Las anotaciones @NotNull y @Min(0)

garantizan que esos valores existan y no sean negativos. Es la base para reglas como compensación por retraso, puntos en vuelos largos y restricción de equipaje en vuelos cortos.

3. Luggage — Representa el equipaje con un único campo: weightKg. También usa @NotNull y @Min(0) para validar el peso. Las reglas de descuento, denegación de ascensos y restricción de equipaje dependen directamente de este valor.

4. BookingRequest — Es la clase de entrada principal: agrupa Passenger, Flight y Luggage, más el booleano emergencyExitSeatAvailable. Las anotaciones @NotNull y @Valid hacen que Spring valide también los objetos anidados. En Drools, este objeto es el “hecho” de entrada que se inserta en la sesión para que las reglas evalúen condiciones como passenger.membershipStatus o flight.delayMinutes.

5. AirlineResponse — Es el objeto de salida donde las reglas escriben los resultados: ascensos, descuentos, compensaciones, puntos de lealtad, acceso VIP, asientos asignados, etc. No lleva anotaciones de validación porque no viene del usuario, sino que lo crea el servicio vacío y Drools lo modifica. El método addMessage() concatena los mensajes de cada regla activada, separados por |.

- Creación de la clase de configuración DroolsConfig.

```
package com.drools.Lab5.config;

import org.kie.api.KieServices;
import org.kie.api.builder.KieBuilder;
import org.kie.api.builder.KieFileSystem;
import org.kie.api.builder.KieModule;
import org.kie.api.runtime.KieContainer;
import org.kie.internal.io.ResourceFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class DroolsConfig {
    private static final KieServices kieServices = KieServices.Factory.get();
    private static final String RULES_AIRLINE_RULES_DRL = "rules/airline_rules.drl";

    @Bean
    public KieContainer kieContainer() {
        KieFileSystem kieFileSystem = kieServices.newKieFileSystem();
        kieFileSystem.write(ResourceFactory.newClassPathResource(RULES_AIRLINE_RULES_DRL));
        KieBuilder kb = kieServices.newKieBuilder(kieFileSystem);
        kb.buildAll();
        KieModule kieModule = kb.getKieModule();
        KieContainer kieContainer =
            kieServices.newKieContainer(kieModule.getReleaseId());
        return kieContainer;
    }
}
```

La clase `DroolsConfig` se utiliza para configurar el motor de reglas de Drools dentro de una aplicación Spring Boot. La anotación `@Configuration` indica que la clase contiene configuraciones administradas por Spring, mientras que `KieServices` funciona como la fábrica principal de Drools para crear los diferentes componentes necesarios. Luego se define la ruta del archivo `credit_rules.drl`, donde se encuentran las reglas de negocio. El método `kieContainer()` está marcado con `@Bean`, lo que permite que Spring gestione automáticamente el objeto `KieContainer` y pueda inyectarlo en otras clases. Dentro del método, primero se crea un `KieFileSystem`, que actúa como un sistema de archivos virtual donde se cargan las reglas. Después, mediante `ResourceFactory.newClassPathResource(...)`, se carga el archivo `.drl` desde la carpeta `resources`. Posteriormente se crea un `KieBuilder`, encargado de leer y compilar las reglas definidas en el archivo DRL. Con `buildAll()` Drools valida y construye todas las reglas, generando un `KieModule`, que representa el módulo compilado de reglas listo para ejecutarse. Finalmente, se crea un `KieContainer` usando el `ReleaseId` del módulo compilado, y este contenedor será el encargado de almacenar y ejecutar las reglas de negocio dentro de la aplicación.

- Creación de la capa servicio.

```
package com.drools.Lab5.service;

import com.drools.Lab5.model.AirlineResponse;
import com.drools.Lab5.model.BookingRequest;
import org.kie.api.runtime.KieContainer;
import org.kie.api.runtime.KieSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class AirlineService {

    @Autowired
    private KieContainer kieContainer;

    public AirlineResponse evaluateBooking(BookingRequest request) {
        AirlineResponse response = new AirlineResponse();
        KieSession kieSession = kieContainer.newKieSession();
        try {
            kieSession.insert(request);
            kieSession.insert(response);
            kieSession.fireAllRules();
        } finally {
            kieSession.dispose();
        }
        if (response.getMessage() == null || response.getMessage().isEmpty()) {
            response.addMessage("No se activaron reglas especiales para esta reserva.");
        }
        return response;
    }
}
```

Servicio de Spring (@Service) que orquesta la evaluación. Inyecta KieContainer con @Autowired y expone el método evaluateBooking(BookingRequest request): crea un AirlineResponse, abre una KieSession, inserta ambos objetos, ejecuta fireAllRules() y cierra la sesión. Si ninguna regla dejó mensaje, agrega uno por defecto.

- Capa de IA

```
package com.drools.Lab5.ai;

import org.springframework.ai.chat.client.ChatClient;
import org.springframework.stereotype.Service;
@Service
public class AIExplanationService {
    private final ChatClient chatClient;
    public AIExplanationService(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }
    public String explain(String decision){
        String prompt = ""
        Eres un experto en reglas de aerolíneas. Explica el siguiente resultado generado

        Resultado:
        %s

        Explica:
        - Qué regla se activó
        - Por qué se activó
        - Qué significa para el pasajero
        """.formatted(decision);

        return chatClient.prompt()
            .user(prompt)
            .call()
            .content();
    }
}
```

AIExplanationService — Servicio opcional (@Service) que usa Spring AI y ChatClient para explicar en lenguaje natural el resultado de Drools. Recibe el mensaje de AirlineResponse en explain(String decision), construye un prompt y devuelve una explicación sobre qué reglas se activaron y qué significan para el pasajero.

- Capa controller

AirlineController — Controlador web (@Controller) con ruta base @RequestMapping("/airline"). Recibe peticiones HTTP, valida con @Valid, delega a AirlineService y devuelve vistas HTML o JSON según el endpoint. Es la capa que conecta el navegador, Postman o cualquier cliente con la lógica de reglas.

```
@Controller
@RequestMapping("/airline")
public class AirlineController {

    @Autowired
    private AirlineService airlineService;

    @Autowired
    private AIExplanationService aiService;

    @PostMapping("/api/evaluate")
    @ResponseBody
    public AirlineResponse evaluateBookingApi(
        @Valid @RequestBody BookingRequest request,
        BindingResult result) {

        if (result.hasErrors()) {
            String errorMessage = result.getAllErrors().stream()
                .map(error -> error.getDefaultMessage())
                .reduce((msg1, msg2) -> msg1 + "; " + msg2)
                .orElse("Errores de validación");

            AirlineResponse response = new AirlineResponse();
            response.addMessage("Error de validación: " + errorMessage);
            return response;
        }

        return airlineService.evaluateBooking(request);
    }
}
```



```

@PostMapping("/evaluateBooking")
@ResponseBody
public Object evaluate(@RequestBody BookingRequest request) {
    AirlineResponse decision = airlineService.evaluateBooking(request);
    String explanation = aiService.explain(decision.getMessage());

    return new Object() {
        public final AirlineResponse result = decision;
        public final String aiExplanation = explanation;
    };
}

@GetMapping("/form")
public String showBookingForm(Model model) {
    BookingRequest bookingRequest = new BookingRequest();
    bookingRequest.setPassenger(new Passenger());
    bookingRequest.setFlight(new Flight());
    bookingRequest.setLuggage(new Luggage());
    bookingRequest.setEmergencyExitSeatAvailable(true);

    model.addAttribute("bookingRequest", bookingRequest);
    return "credit_form";
}

```

```

@PostMapping("/evaluate")
public String evaluateBookingWeb(
    @Valid BookingRequest bookingRequest,
    BindingResult result,
    Model model) {

    if (result.hasErrors()) {
        return "credit_form";
    }

    AirlineResponse airlineResponse = airlineService.evaluateBooking(bookingRequest);
    model.addAttribute("airlineResponse", airlineResponse);
    return "credit_result";
}

```

Hay 4 endpoints porque cubren 3 formas distintas de usar la misma lógica:

GET /airline/form

Muestra el formulario HTML al usuario en el navegador

POST /airline/evaluate

Recibe el formulario y devuelve la página credit_result.html

POST /airline/api/evaluate

Para Postman u otras apps: envía JSON y recibe JSON puro

POST /airline/evaluateBooking

Igual que la API, pero agrega una explicación generada por IA

- Reglas Drools y pruebas es Postman

- **Regla 1 — UpgradeToBusinessClassForFrequentFlyersWithDelays**

Si el pasajero tiene membresía Gold o Platinum y el vuelo tiene un retraso superior a 60 minutos, se le otorga ascenso a clase ejecutiva, siempre que siga siendo elegible para ascensos.

```
1 {
2   "passenger": {
3     "name": "Ana López",
4     "age": 35,
5     "membershipStatus": "Gold",
6     "travelingWithChildren": false,
7     "seatPreference": "window"
8   },
9   "flight": {
10    "delayMinutes": 90,
11    "durationHours": 3
12  },
13  "luggage": {
14    "weightKg": 15
15  },
16  "emergencyExitSeatAvailable": false
17 }
```

body Cookies Headers Test Results 1/1 200 OK - 21 ms - 537 B Save Response

JSON Preview Visualize

```
1 {
2   "businessClassUpgrade": true,
3   "priorityCheckIn": false,
4   "discountPercentage": 0.0,
5   "upgradeEligible": true,
6   "assignedSeat": null,
7   "emergencyExitSeatOccupied": false,
8   "compensationAmount": 0.0,
9   "loyaltyPointsAdded": 0,
10  "luggageAllowed": true,
11  "vipLoungeAccess": false,
12  "preferentialFamilySeat": false,
13  "message": "Ascenso a clase ejecutiva por estatus frecuente y retraso superior a 60 minutos."
14 }
```

- **Regla 2 — PriorityCheckInForSeniors**

Si el pasajero tiene más de 65 años, se activa automáticamente el check-in prioritario para facilitar su proceso en el aeropuerto.

```
{
  "passenger": {
    "name": "Pedro Sánchez",
    "age": 45,
    "membershipStatus": "Basic",
    "travelingWithChildren": false,
    "seatPreference": "Aisle"
  },
  "flight": {
    "delayMinutes": 10,
    "durationHours": 2
  },
  "luggage": {
    "weightKg": 12
  },
  "emergencyExitSeatAvailable": false
}
```

Cookies Headers 5 Test Results 1/1 ↻

200 OK • 21 ms

▼ ▶ Preview 🖼 Visualize ▼

```
{
  "businessClassUpgrade": false,
  "priorityCheckIn": false,
  "discountPercentage": 0.0,
  "upgradeEligible": true,
  "assignedSeat": null,
  "emergencyExitSeatOccupied": false,
  "compensationAmount": 0.0,
  "loyaltyPointsAdded": 0,
  "luggageAllowed": true,
  "vipLoungeAccess": false,
  "preferentialFamilySeat": false,
  "message": "No se activaron reglas especiales para esta reserva."
}
```

- **Regla 3 — DiscountForLightLuggage**

Si el equipaje pesa menos de 10 kg, se aplica un descuento del 10% sobre la reserva como beneficio por viajar con equipaje ligero.

```
{
  "passenger": {
    "name": "Diego Torres",
    "age": 28,
    "membershipStatus": "Basic",
    "travelingWithChildren": false,
    "seatPreference": "Window"
  },
  "flight": {
    "delayMinutes": 0,
    "durationHours": 3
  },
  "luggage": {
    "weightKg": 12
  },
  "emergencyExitSeatAvailable": false
}
```

Cookies Headers 5 Test Results 1/1 ↺

200 OK • 27 n

⌵ ▾ ▶ Preview 🖼 Visualize ▾

```
{
  "businessClassUpgrade": false,
  "priorityCheckIn": false,
  "discountPercentage": 0.0,
  "upgradeEligible": true,
  "assignedSeat": null,
  "emergencyExitSeatOccupied": false,
  "compensationAmount": 0.0,
  "loyaltyPointsAdded": 0,
  "luggageAllowed": true,
  "vipLoungeAccess": false,
  "preferentialFamilySeat": false,
  "message": "No se activaron reglas especiales para esta reserva."
}
```

- **Regla 4 — DenyUpgradeForOverweightLuggage**

Si el equipaje supera los 23 kg, el pasajero queda marcado como no elegible para ascensos de clase, independientemente de su membresía.

```
{
  "passenger": {
    "name": "Jorge Vargas",
    "age": 40,
    "membershipStatus": "Platinum",
    "travelingWithChildren": false,
    "seatPreference": "Aisle"
  },
  "flight": {
    "delayMinutes": 120,
    "durationHours": 4
  },
  "luggage": {
    "weightKg": 25
  },
  "emergencyExitSeatAvailable": false
}

cookies Headers Test Results 1/1 200 OK - 28 ms - 659 B Save Response
Preview Visualize
"businessClassUpgrade": true,
"priorityCheckIn": false,
"discountPercentage": 8.8,
"upgradeEligible": false,
"assignedSeat": null,
"emergencyExitSeatOccupied": false,
"compensationAmount": 8.8,
"loyaltyPointsAdded": 0,
"luggageAllowed": true,
"vipLoungeAccess": true,
"preferentialFamilySeat": false,
"message": "Ascenso a clase ejecutiva por estatus frecuente y retraso superior a 60 minutos. | Pasajero no elegible para ascensos por equipaje mayor a 23 kg. | Acceso al salón VIP otorgado por membresía Platinum."
```

- **Regla 5 — AssignEmergencyExitSeatToYoungAdults**

Si el pasajero tiene entre 18 y 40 años, su preferencia de asiento es "Any" y hay un asiento de salida de emergencia disponible, se le asigna ese asiento y se marca como ocupado.

```
{
  "passenger": {
    "name": "Camila Ortiz",
    "age": 38,
    "membershipStatus": "Basic",
    "travelingWithChildren": false,
    "seatPreference": "Any"
  },
  "flight": {
    "delayMinutes": 8,
    "durationHours": 3
  },
  "luggage": {
    "weightKg": 12
  },
  "emergencyExitSeatAvailable": true
}

cookies Headers Test Results 1/1 200 OK - 32 ms - 536 B
Preview Visualize
"businessClassUpgrade": false,
"priorityCheckIn": false,
"discountPercentage": 8.8,
"upgradeEligible": true,
"assignedSeat": "Salida de Emergencia",
"emergencyExitSeatOccupied": true,
"compensationAmount": 0.0,
"loyaltyPointsAdded": 0,
"luggageAllowed": true,
"vipLoungeAccess": false,
"preferentialFamilySeat": false,
"message": "Asiento de salida de emergencia asignado y marcado como ocupado."
```

- **Regla 6 — CompensationForExtremeDelays**

Si el vuelo tiene un retraso mayor a 180 minutos (3 horas), se otorga una compensación económica de \$200 al pasajero.

```
{
  "passenger": {
    "name": "Lucía Pérez",
    "age": 50,
    "membershipStatus": "Basic",
    "travelingWithChildren": false,
    "seatPreference": "Aisle"
  },
  "flight": {
    "delayMinutes": 120,
    "durationHours": 4
  },
  "luggage": {
    "weightKg": 15
  },
  "emergencyExitSeatAvailable": false
}
```

cookies Headers 5 Test Results 1/1

200 OK

Preview Visualize

```
"businessClassUpgrade": false,
"priorityCheckIn": false,
"discountPercentage": 0.0,
"upgradeEligible": true,
"assignedSeat": null,
"emergencyExitSeatOccupied": false,
"compensationAmount": 0.0,
"loyaltyPointsAdded": 0,
"luggageAllowed": true,
"vipLoungeAccess": false,
"preferentialFamilySeat": false,
"message": "No se activaron reglas especiales para esta reserva."
```

- **Regla 7 — ExtraLoyaltyPointsForLongFlights**

Si el pasajero tiene membresía distinta a Basic y el vuelo dura más de 5 horas, se agregan 500 puntos de lealtad a su cuenta.

```
{
  "passenger": {
    "name": "Ricardo Gómez",
    "age": 42,
    "membershipStatus": "Gold",
    "travelingWithChildren": false,
    "seatPreference": "Window"
  },
  "flight": {
    "delayMinutes": 0,
    "durationHours": 6
  },
  "luggage": {
    "weightKg": 14
  },
  "emergencyExitSeatAvailable": false
}

[{"businessClassUpgrade": false,
"priorityCheckIn": false,
"discountPercentage": 0.6,
"upgradeEligible": true,
"assignedSeat": null,
"emergencyExitSeatOccupied": false,
"compensationAmount": 0.0,
"loyaltyPointsAdded": 500,
"luggageAllowed": true,
"vipLoungeAccess": false,
"preferentialFamilySeat": false,
"message": "500 puntos de lealtad agregados por vuelo largo con membresia premium."}]
```

- **Regla 8 — RestrictLuggageOnShortFlights**

Si el vuelo dura menos de 2 horas y el equipaje pesa más de 15 kg, el equipaje se marca como no permitido para ese vuelo.

```
{
  "passenger": {
    "name": "Natalia Rojas",
    "age": 33,
    "membershipStatus": "Basic",
    "travelingWithChildren": false,
    "seatPreference": "Aisle"
  },
  "flight": {
    "delayMinutes": 0,
    "durationHours": 1.5
  },
  "luggage": {
    "weightKg": 18
  },
  "emergencyExitSeatAvailable": false
}

[{"businessClassUpgrade": false,
"priorityCheckIn": false,
"discountPercentage": 0.0,
"upgradeEligible": true,
"assignedSeat": null,
"emergencyExitSeatOccupied": false,
"compensationAmount": 0.0,
"loyaltyPointsAdded": 0,
"luggageAllowed": true,
"vipLoungeAccess": false,
"preferentialFamilySeat": false,
"message": "No se activaron reglas especiales para esta reserva."}]
```

- **Regla 9 — VipLoungeAccessForPlatinumMembers**

Si el pasajero tiene membresía Platinum, se le otorga acceso al salón VIP del aeropuerto.

```
{
  "passenger": {
    "name": "Patricia Londoño",
    "age": 38,
    "membershipStatus": "Platinum",
    "travelingWithChildren": false,
    "seatPreference": "Window"
  },
  "flight": {
    "delayMinutes": 0,
    "durationHours": 3
  },
  "luggage": {
    "weightKg": 12
  },
  "emergencyExitSeatAvailable": false
}
```

cookies Headers 5 Test Results 1/1 200 OK

Preview Visualize

```
{
  "businessClassUpgrade": false,
  "priorityCheckIn": false,
  "discountPercentage": 0.0,
  "upgradeEligible": true,
  "assignedSeat": null,
  "emergencyExitSeatOccupied": false,
  "compensationAmount": 0.0,
  "loyaltyPointsAdded": 0,
  "luggageAllowed": true,
  "vipLoungeAccess": true,
  "preferentialFamilySeat": false,
  "message": "Acceso al salón VIP otorgado por membresía Platinum."
}
```

- **Regla 10 — PreferentialSeatForFamilies**

Si el pasajero viaja con niños y no tiene una preferencia específica de asiento (Any), se le asigna un asiento preferencial para familias.

```
{
  "passenger": {
    "name": "Claudia Restrepo",
    "age": 36,
    "membershipStatus": "Basic",
    "travelingWithChildren": true,
    "seatPreference": "Any"
  },
  "flight": {
    "delayMinutes": 0,
    "durationHours": 3
  },
  "luggage": {
    "weightKg": 14
  },
  "emergencyExitSeatAvailable": false
}
```

cookies Headers 5 Test Results 1/1

Preview Visualize

```
{
  "businessClassUpgrade": false,
  "priorityCheckIn": false,
  "discountPercentage": 0.0,
  "upgradeEligible": true,
  "assignedSeat": "Asiento Familiar Preferencial",
  "emergencyExitSeatOccupied": false,
  "compensationAmount": 0.0,
  "loyaltyPointsAdded": 0,
  "luggageAllowed": true,
  "vipLoungeAccess": false,
  "preferentialFamilySeat": true,
  "message": "Asiento preferencial para familias asignado."
}
```


- Vistas html

- **credit_form.html — Formulario de entrada**

Es la vista web donde el usuario ingresa los datos de la reserva: información del pasajero (nombre, edad, membresía, preferencia de asiento, si viaja con niños), datos del vuelo (retraso y duración), peso del equipaje y si hay asiento de salida de emergencia disponible. Al enviarlo con el botón "Evaluar Reserva", los datos se envían por POST a /airline/evaluate y Spring los mapea automáticamente al objeto BookingRequest.

- **credit_result.html — Formulario de resultados**

Es la vista que muestra el resultado después de ejecutar las reglas Drools. Presenta de forma organizada los beneficios otorgados (ascenso, check-in prioritario, descuentos, compensaciones, puntos, VIP), las restricciones aplicadas (equipaje no permitido, no elegible para ascensos), el asiento asignado si corresponde, y un resumen textual de todas las reglas que se activaron. Incluye un enlace para volver al formulario y realizar una nueva evaluación.

5. Conclusiones

- Drools permite implementar reglas de negocio de manera organizada y desacoplada del código principal de la aplicación.
- La integración entre Spring Boot y Drools facilita la creación de aplicaciones empresariales dinámicas y escalables.
- Las reglas DRL permiten expresar condiciones y acciones de forma declarativa y fácil de mantener.
- El uso de Spring Validation ayuda a garantizar que los datos recibidos sean válidos antes de procesarse.
- Las pruebas realizadas con Postman permitieron verificar el correcto funcionamiento de cada regla definida en el sistema.
- La integración con inteligencia artificial mejora la experiencia del usuario al proporcionar explicaciones claras sobre las decisiones tomadas por el sistema.

6. Bibliografía

<https://docs.drools.org/latest/drools-docs/drools/introduction/index.html>

<https://www.thymeleaf.org/>